The Family Metro Map Layout for General Graphs

Spanning tree, metro routing of cycle edges, overlap repair and complexity

Technical description of `family_metro_map_graph.js`
(companion to the document on the tree algorithm, `familymetromap.js`)

Abstract

This document describes how the application draws an arbitrary undirected graph (cycles and several components allowed) with at most eight neighbours per vertex as a London-Underground-style metro map. Every edge runs along one of the eight octilinear directions and is either straight or has exactly one 45° bend, and the edges at a vertex leave in different directions. The graph is split into a breadth-first spanning forest, whose edges are placed by the beam search of the tree algorithm, and the remaining *cycle edges*, which are routed as metro links at the moment their second endpoint is placed. Because the cost of these routes is part of the placement score, vertices are placed where their cycles can close without crossings. A search over layout settings, overlap detection (incremental in step-by-step mode) and the Fix buttons complete the method. We state the constraints on the input (degree, self-loops, planarity), give every quantity used by the code, analyse the running time, and report experiments on grids, cycles, trees with extra edges, random graphs and K_5.

Contents

1 Overview **1**

2 Input and constraints **2**

3 Spanning forest and tree edges **3**

4 Cycle edges as metro links **4**

5 Several components **6**

6 Settings search **7**

7 Overlap detection **7**

8 Overlap repair (The Fix buttons) **8**

9 Time and space complexity **9**

10 Experiments **10**

11 Limitations **12**

1 Overview

The input is an undirected graph $G = (V, E)$ with $n = |V|$ vertices, $m = |E|$ edges and c connected components, a chosen root r, a shrinking factor $\alpha \in (0, 1]$, and a gender $\sigma(v) \in \{\mathrm{F}, \mathrm{M}\}$ for every vertex (derived

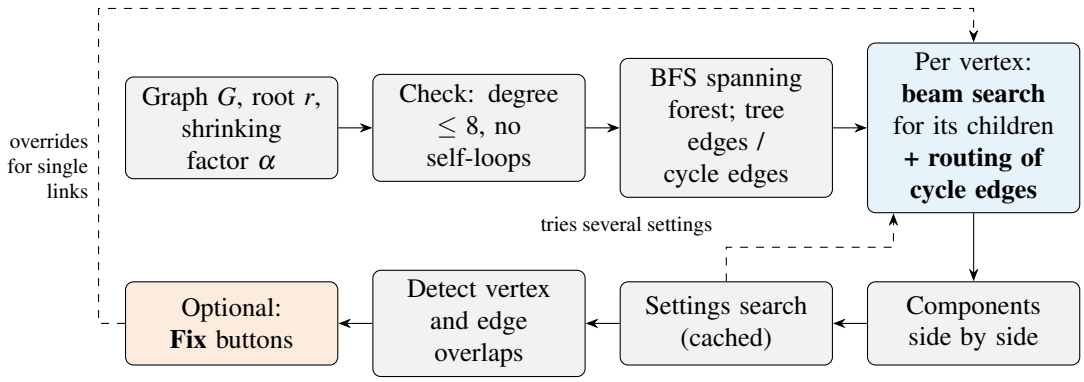


Figure 1: Pipeline. The layout (blue) is deterministic. The settings search and the Fix buttons never move vertices directly: they choose parameters or record *overrides* for single links and re-run the layout, so cycle edges are always re-routed consistently.

from the vertex label: an even character code of the first letter means female). The output is a position for every vertex and at most one bend point for every edge.

Figure 2 shows a typical result. Edges of the spanning tree (grey) are drawn exactly as in the tree algorithm; cycle edges (green, dashed in all figures of this document; in the application all edges have the same colour) are metro links between two already placed vertices.

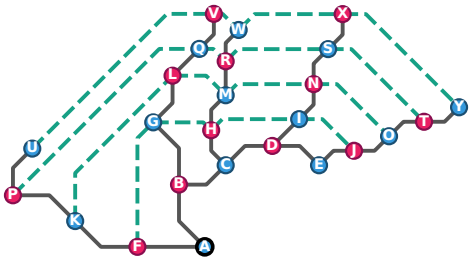


Figure 2: A 5×5 grid graph ($n = 25$, $m = 40$, 16 cycle edges), root A (black border), $\alpha = 0.75$. Grey: BFS spanning tree; green dashed: cycle edges. No vertex overlaps and no colliding edges.

2 Input and constraints

property	allowed?	remark
cycles	yes	drawn as metro links (section 4)
several components	yes	laid out separately and placed side by side (section 5)
isolated vertices	yes	a component with one vertex
degree $\deg(v) \leq 8$	**required**	checked before every layout; otherwise an error names the vertices
self-loops	no	rejected (a loop cannot be drawn as a metro link)
parallel edges	no	the input formats (adjacency list / matrix) cannot express them
planarity	not required	but non-planar graphs always keep crossings (section 2.2)

2.1 Why at most eight neighbours

A metro map has eight directions $k \cdot 45°$, $k = 0, \ldots, 7$, and two edges that leave a vertex in the same direction share their first segment, which is reported as an edge overlap. A vertex can therefore have at most eight edges without a forced overlap. For a non-root vertex one direction is taken by the edge from its BFS parent, leaving seven for the remaining edges; the root has all eight available. The bound is necessary but

not sufficient: the direction of a cycle edge is dictated by the positions of its endpoints (section 4.1), so two cycle edges can still compete for the same direction even when $\deg(v) \leq 8$. Such conflicts are penalised during placement and are the target of the Fix buttons; when they cannot be removed they are reported.

2.2 Planarity

The algorithm does *not* test planarity and works for every graph that satisfies the degree bound. Planarity matters for what can be achieved:

- A **non-planar** graph, for example K_5 or $K_{3,3}$, cannot be drawn without crossings in any style; at least its crossing number $\mathrm{cr}(G)$ of crossings remains ($\mathrm{cr}(K_5) = \mathrm{cr}(K_{3,3}) = 1$). These crossings appear in the Overlapping / Touching Edges panel and cannot be removed by the Fix buttons.
- A **planar** graph has a crossing-free straight-line drawing (Fáry's theorem), but the metro style restricts the drawing much more: octilinear segments, at most one bend per edge, distinct directions at each vertex, and tree edges with gender-dependent shapes and prescribed lengths. The algorithm is a heuristic and does not guarantee a crossing-free result for every planar graph. In the experiments (section 10) all grids and cycles were drawn without any overlap.
- The method also does not aim for the minimum number of crossings, which is NP-hard to compute in general; it reduces collisions greedily and through the settings search and the Fix buttons.

Figure 3 contrasts the planar K_4 with the non-planar K_5.

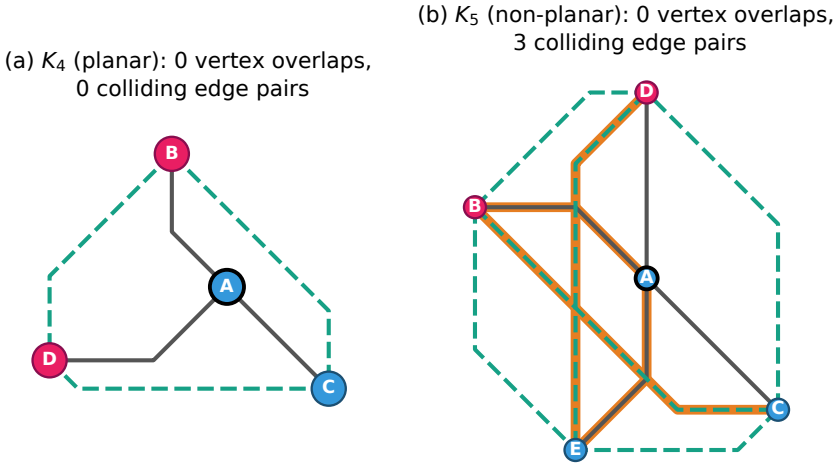


Figure 3: Complete graphs ($\alpha = 0.75$). (a) K_4 is planar and is drawn without collisions. (b) K_5 is not planar, so crossings cannot be avoided; colliding edges are highlighted in orange (here 3 colliding edge pairs, against the theoretical minimum of one crossing for K_5).

3 Spanning forest and tree edges

Spanning forest. A breadth-first search from the root r visits its component and fixes, for every vertex $v \neq r$, a BFS parent $p(v)$; every other component is searched from its first vertex in input order. The *tree edges* $\{v, p(v)\}$ form a spanning forest with $n - c$ edges; the remaining

$$k \;=\; m - n + c$$

edges are the *cycle edges*. BFS keeps cycle edges short in terms of generations: the endpoints of a cycle edge are in the same or in neighbouring BFS layers, so they are usually drawn close to each other.

Tree edges. Tree edges are placed exactly as in the tree algorithm (see the companion document). Briefly: each vertex v has an orientation $k(v) \in \{0,\dots,7\}$, even for female and odd for male vertices. The link $p(v) \to v$ leaves $p(v)$ in direction $(k(p(v)) + g) \bmod 8$ with branch offset $g \in \{0, \pm 2, \pm 1, \pm 3, 4\}$; it is straight if that direction has the right parity for v and otherwise has one 45° bend $s = \pm 1$ with two segments of length $\ell/\sqrt{2+\sqrt{2}}$. The length is $\ell = \max(L_0\,\alpha^{\gamma(v)}, 34\text{ px})$ with the line generation γ. Vertices are processed in BFS order, and the links to all children of a vertex are chosen together by a beam search of width B (4 by default). A candidate is scored by

$$\text{score}_{\text{tree}} = 10^6 V + 10^5 E + 40\max(0,\, 0.9\ell - c) + P_{\text{room}} + 12\,\pi(g) - 0.02\,\bar{\delta},$$

with vertex conflicts V, edge conflicts E, clearance c, look-ahead room P_{room}, branch preference $\pi(g)$ and average distance $\bar{\delta}$. The general-graph version adds two terms to this score (section 4.4) and reserves the directions used by cycle edges (section 4.5). If the graph is a tree there are no cycle edges, both additions vanish, and the layout is identical to the one of the tree algorithm (verified on 378 test trees, including trees with more than 100 vertices).

4 Cycle edges as metro links

4.1 Shapes

A cycle edge joins two vertices w and v that are both placed when it is routed, so its shape is determined by the vector $\mathbf{d} = p_v - p_w = (d_x, d_y)$. The direction of $\mathbf{d}$ lies in a 45° sector between two neighbouring metro directions $d_1 = \lfloor \text{atan2}(d_y, d_x)/(\pi/4) \rfloor \bmod 8$ and $d_2 = d_1 + 1 \bmod 8$. Writing $\mathbf{d}$ in the basis $\mathbf{e}_{d_1}, \mathbf{e}_{d_2}$,

$$\mathbf{d} = s_1\,\mathbf{e}_{d_1} + s_2\,\mathbf{e}_{d_2}, \qquad s_1 = \frac{d_x e_{2,y} - d_y e_{2,x}}{\det(\mathbf{e}_{d_1}, \mathbf{e}_{d_2})}, \qquad s_2 = \frac{e_{1,x} d_y - e_{1,y} d_x}{\det(\mathbf{e}_{d_1}, \mathbf{e}_{d_2})}, \qquad s_1, s_2 \geq 0,$$

with $\det(\mathbf{e}_{d_1}, \mathbf{e}_{d_2}) = \sin 45°$. This gives exactly two metro shapes with one bend (fig. 4):

- **shape A:** first s_1 along d_1, then s_2 along d_2; bend at $p_w + s_1 \mathbf{e}_{d_1}$;
- **shape B:** first s_2 along d_2, then s_1 along d_1; bend at $p_w + s_2 \mathbf{e}_{d_2}$.

Both turn by 45° (interior angle 135°), as on the London Underground map, and both have length $s_1 + s_2$. If $\min(s_1, s_2) < 1$ px the second segment would be invisible, and the edge is drawn **straight** along the dominant direction. No other shapes are used: no right angles, no acute angles and never more than one bend.

The *ports* of a shape are the directions in which it leaves its endpoints: shape A leaves w in direction d_1 and arrives at v in direction d_2, i.e. it occupies port d_1 at w and port $(d_2 + 4) \bmod 8$ at v; for shape B the roles of d_1 and d_2 are swapped.

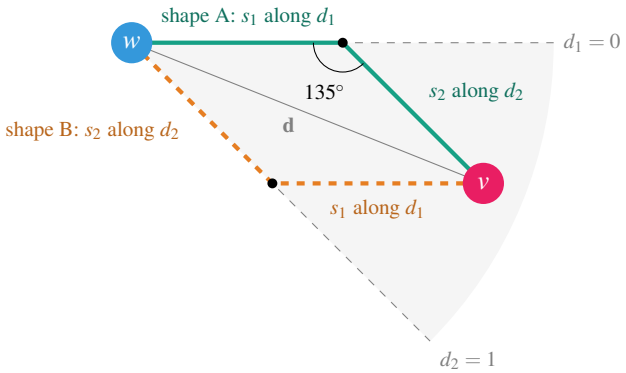


Figure 4: The two metro shapes of a cycle edge w–v. The vector $\mathbf{d}$ lies in the sector between directions d_1 and d_2 (screen coordinates, y pointing down); shape A uses ports d_1 at w and $d_2 + 4$ at v, shape B ports d_2 at w and $d_1 + 4$ at v.

4.2 When a cycle edge is routed

Lemma 1. *Every cycle edge is routed exactly once, at the moment its second endpoint is placed, and both endpoints are placed at that moment.*

Reason. Vertices are placed in BFS order, children of one parent together in one beam search. When a candidate position for v is evaluated, each cycle neighbour w of v is either already placed (committed earlier, or placed earlier in the same beam state as a sibling of v) or not yet placed. In the first case the edge is routed now; in the second case v will be placed before w, so the edge is routed when w is placed. □

4.3 Route cost

For each of its (at most two) shapes, a cycle edge is scored against all geometry known at that point (committed vertices and segments, the tentative siblings of the beam state, the candidate's own tree link and the cycle edges of v routed just before):

$$\text{cost}(\text{shape}) = 10^6\, V_r + 10^5\, E_r + 12\,[\text{bent}] + 4\max(0,\ 16 - \min(s_1, s_2)),$$

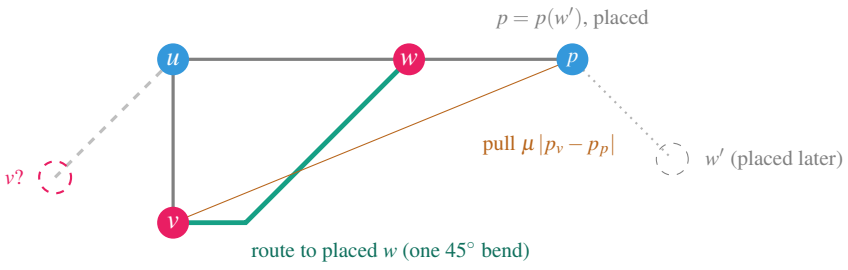
where V_r counts (vertex, segment) pairs closer than 12 px for vertices other than v and w (the edge would run through a vertex) and E_r counts pairs (route segment, existing segment) that collide by the edge predicate of section 7: a proper crossing, a touch closer than 6 px, or two segments leaving a shared endpoint in the same direction. The last case is exactly a *port conflict*, so ports are checked by the same predicate. The bend penalty 12 has the same scale as one step of the branch preference of tree links, and the last term discourages a bend within 16 px of a vertex. The cheaper shape is kept; ties keep shape A.

4.4 Integration into the beam search

The beam search of the tree algorithm is extended in EXPAND: for every candidate option o of a child v,

$$\text{score}(o) \;=\; \text{score}_{\text{tree}}(o) \;+\; \sum_{w \in C_{\text{placed}}(v)} \text{cost}\big(\text{best route } w\text{–}v\big) \;+\; \mu \sum_{w \in C_{\text{later}}(v)} \big|p_v - p_{p(w)}\big|,$$

where $C_{\text{placed}}(v)$ are the cycle neighbours of v that are already placed and $C_{\text{later}}(v)$ those that are not yet placed but whose BFS parent $p(w)$ is (the parent's position is where w will appear). The second sum is a *pull* with weight $\mu = 1$ per pixel: it keeps v close to where its future cycle neighbours will be, so that their cycle edges stay short. Figure 5 illustrates the three kinds of neighbours.



candidate positions of v (child of u): solid = option being scored, dashed = another option

Figure 5: Scoring one candidate position of v (child of u). The cycle edge to the placed neighbour w is routed immediately (green) and its cost is added. The not-yet-placed cycle neighbour w' will be placed next to its BFS parent p, so the distance from v to p is added as a pull. Unplaced neighbours whose parent is not placed yet contribute nothing.

When the best state is committed, the cycle edges stored with it become part of the drawing: their segments are added to the list of drawn segments (so every later placement avoids them) and their bends are stored for drawing.

Algorithm 1 EXPAND with cycle edges (for child v of u; beam $\mathscr{B}$, options O)

for all states $S \in \mathscr{B}$ **do**
  $used \leftarrow usedPorts(u) \cup \{\text{back direction of } u\} \cup \{\text{first directions of the children in } S\}$
  for all options $o \in O$ with first direction $\notin used$ **do**
    place v by o (tree link, as in the tree algorithm); $c \leftarrow \text{score}_{\text{tree}}(o)$; $own \leftarrow$ tree-link segments
    for all cycle neighbours w of v **do**
      if w is placed (committed or in S) **then**
        $R \leftarrow$ cheaper of the shapes of w–v (section 4.3), checked against committed geometry, S and own
        $c \leftarrow c + \text{cost}(R)$; $own \leftarrow own \cup R$
      else if $p(w)$ is placed **then**
        $c \leftarrow c + \mu\,|p_v - p_{p(w)}|$
    add $(S \cup \{(o, \text{routes})\},\ \text{cost}(S) + c)$ to the result
if no option had a free direction, repeat without the direction test (the conflict is then penalised as an edge collision)

4.5 Port reservation

For every placed vertex the algorithm keeps the set $usedPorts(x)$ of directions already used at x: the arriving tree link, the links to its children and the ends of its cycle edges. When the children of u are placed, every direction in $usedPorts(u)$ is blocked for their first segments, exactly like the back direction in the tree algorithm. In particular, if a cycle edge reached u after u was placed but before its children were, the children avoid that direction. For a tree, $usedPorts(u)$ contains only the arriving direction, so nothing changes. The ports of cycle edges themselves cannot be chosen freely (they follow from the geometry), so conflicts at their endpoints are penalised through E_r rather than forbidden.

5 Several components

Each connected component is laid out on its own, from the root for the root's component and from its first vertex for the others. The drawings are then placed from left to right in that order: each component is shifted so that its leftmost point is 120 px to the right of everything drawn so far, and its top is aligned with the top of the drawing so far (fig. 6). The BFS layer display and the step-by-step order merge the components layer by layer (layer d holds every vertex at depth d in its own component). For a connected graph nothing is shifted, so the layout is unchanged.

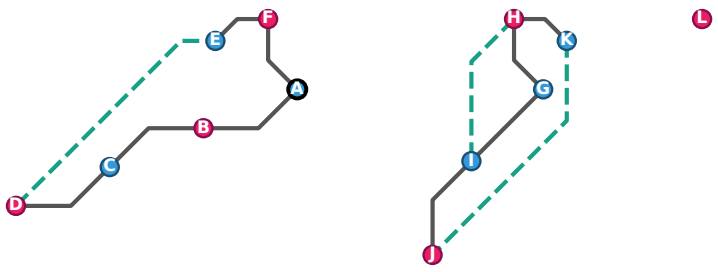


Figure 6: A graph with three components: a 6-cycle (root A), a 5-vertex component with two cycles, and an isolated vertex L.

6 Settings search

The layout has three global settings: the starting edge length L_0 (90 px for up to 100 vertices, then $+30$ px per further band of 25 vertices), the beam width B, and an optional extra shrink per generation (effective α is the user's α times a factor). The search runs whenever the graph has a cycle (and, as in the tree version, for trees with more than 100 vertices). It tries candidate settings in five stages and keeps the layout with the fewest overlapping vertex pairs, then the fewest colliding edge pairs:

extra shrink	candidates (length factor f times L_0, beam width B), shortest first
$\times 1$ (user's α)	$(1,4)$, $(1,1)$, $(1,2)$, $(1.25,2)$, $(1.5,2)$, $(1.75,2)$, $(2,1)$, $(2.5,1)$, $(3,2)$
$\times 0.8$	$(1.5,2)$, $(1.75,2)$, $(3,1)$, $(3,2)$
$\times 0.7$	$(1.25,1)$, $(1.5,1)$, $(1.75,1)$
$\times 0.6$	$(2,1)$, $(2.5,1)$, $(2.5,2)$
$\times 0.5$	$(2,1)$, $(2.5,2)$, $(3,1)$, $(3,2)$

The search stops at the first candidate without any overlap. Because the default setting $(1,4)$ is tried first, the result is never worse than a single run, the user's α is kept whenever that is enough, and among clean layouts the most compact one is chosen. Vertex overlaps are counted with a threshold of 17.5 px in the search (16 px plus the effect of rounding positions to whole pixels on screen). The chosen setting is cached per graph, root and α, so Run Full Metro Layout, Step by Step BFS Draw, the overlap panels and the Fix buttons all use the same layout; the status line reports which setting was used.

7 Overlap detection

- **Vertex overlaps:** two visible vertices whose on-screen positions are closer than 16 px (the disk size).
- **Edge collisions:** every pair of segments of different edges (tree or cycle) is tested. Two segments collide if they cross, if they leave a shared endpoint in the same direction (angle difference below 0.05 rad), or if they come closer than 6 px without sharing an endpoint. Segments of the same edge are never compared.

Pairs are grouped into connected components, so that mutually overlapping vertices or edges are listed once ("Vertices B, H, N overlap", "Edges A-B, B-C collide / overlap"). A group of edges is reported as *collide / overlap* if at least one of its pairs crosses or shares a start direction, and as *touch* otherwise. Groups are listed in a fixed order: by their first member, with vertices in input order and edges in alphabetical order, and the members of a group in the same order.

7.1 Incremental checking in Step by Step BFS Draw

Step by Step BFS Draw computes the complete layout once, on its first click, and then reveals one vertex per click in BFS order. The overlap lists must show exactly what the full check would report for the part drawn so far, and mark groups that appeared in the last click as NEW. Recomputing them from scratch after every click (all vertex pairs, all segment pairs, and in the earlier version even the layout itself) costs $O(n^2)$ per click and $O(n^3)$ for a click-through. They are therefore updated incrementally:

1. **Stored data (first click).** The whole-pixel screen position of every vertex, the segments of every edge with their index in the segment list, and for every vertex the edges incident to it.
2. **New vertex.** When vertex x is revealed it is compared with the vertices already drawn only: $O(n)$.
3. **New edges.** The edges that become visible are the edges from x to drawn vertices (at most $\deg x \leq 8$). Each is compared with the edges already drawn only, $O(m)$ per click. For a pair of edges the segment pairs (at most 4) are tested in the order of the full check (by index in the segment list), so the first colliding pair gives the same *touch* / *overlap* type.

4. **Groups.** Overlapping vertices and colliding edges are kept in two union-find structures (union by size, path compression). Every group stores its members sorted in the listing order (two groups are merged by a linear merge of their sorted lists), the sums of the vertex coordinates (for the "overlap at approx (x,y)" position) and whether it contains a real overlap. Because vertices and edges are only ever added and positions never change, groups only grow or merge and never split.
5. **NEW.** A group is new in a click exactly when it was created or grew in that click, so the groups touched by the unions of the click are marked; no comparison with the previous lists is needed.
6. **Listing.** The lists are written by one pass over all vertices and one pass over all edges in listing order, printing each group at its first member: $O(n+m)$ per click.

The lists produced this way were compared with the old implementation after every click in 15 runs on 14 graphs (679 clicks, including graphs with up to 6 groups of overlapping vertices and 5 groups of colliding edges): members, types, positions and NEW marks were identical in every click.

8 Overlap repair (The Fix buttons)

8.1 Prevention during the layout

Most overlaps are prevented while the layout is built, by the terms of the placement score: vertex conflicts (10^6) and edge conflicts (10^5) of tree links, the same two terms for every routed cycle edge, the pull towards future cycle neighbours, port reservation, and finally the settings search, which compares complete layouts.

8.2 The Fix buttons

The two buttons run a local search over *overrides*: choices that the layout must respect when it is run again.

- a forced branch offset g and/or bend sign s for one tree link (the paper's "flipping of sub-trees");
- a length factor (0.76–1.3) for one tree link;
- a local shrinking factor (0.8, 0.65 or 0.5) for the subtree below a vertex (a local version of Lemma 4.3 of Korst et al.).

After each change the whole layout is re-run, so the vertices below the modified link move and **all cycle edges are re-routed** with the rules of section 4; cycle edges therefore never need overrides of their own.

Algorithm 2 FIX(mode), mode $\in$ {vertex, edge}

1: $best \leftarrow$ current overrides; $e \leftarrow$ EVALUATE($best$)
2: **for** at most 16 rounds, while overlaps of this kind remain and the budget is not used up **do**
3: $\quad\mathscr{C} \leftarrow$ vertices involved in the overlaps (vertex mode), or the child end of every colliding tree edge and **both ends of every colliding cycle edge** (edge mode); add their BFS parents and grandparents
4: $\quad$**for all** $x \in \mathscr{C}$ with parent p **do** try every (g,s) for link $p \to x$; keep the best if it lowers the cost
5: $\quad$**if** nothing improved **then** try length factors for one link
6: $\quad$**if** nothing improved **then** try local shrinking of one subtree
7: $\quad$**if** nothing improved **then stop**

EVALUATE runs the layout and returns

$$\text{cost} = 100\,000 \cdot (\#\text{vertex overlaps}) + 5\,000 \cdot (\#\text{colliding edge pairs}) - (\text{minimum vertex distance}).$$

Fix Edge Overlaps never accepts a change that adds vertex overlaps. The number of evaluations is limited to $\max(150, \min(500, \lfloor 25\,000/n \rceil))$. Every accepted change is listed in Summary of Changes; overlaps that remain are listed in red. Figure 7 shows the effect on a tree with additional cycle edges.

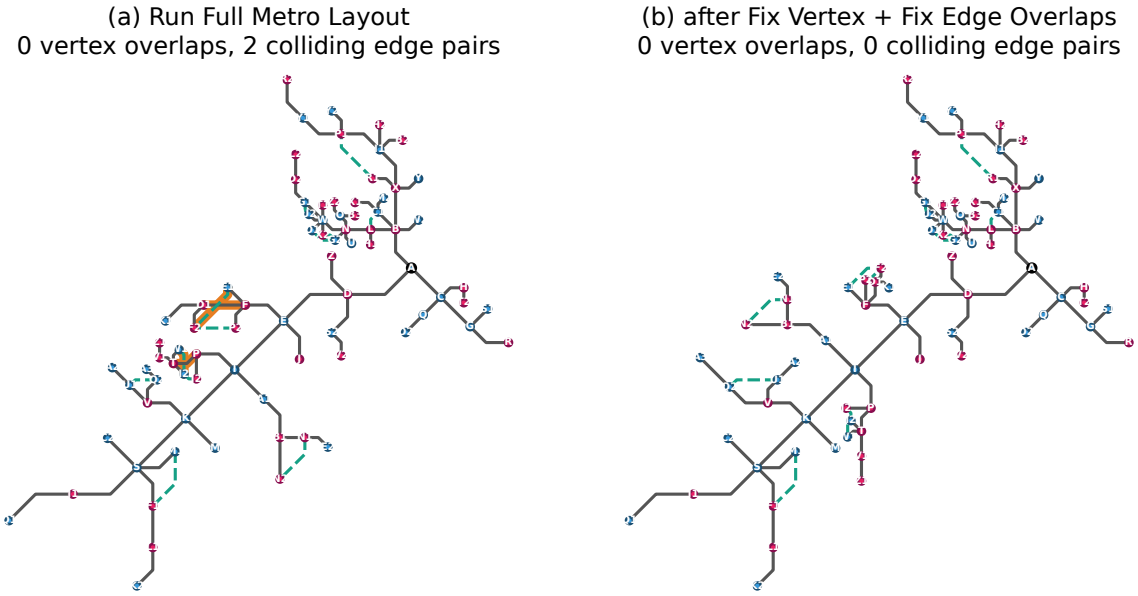


Figure 7: An 80-vertex tree with 12 additional cycle edges ($\alpha = 0.75$). (a) After Run Full Metro Layout two pairs of edges collide (orange). (b) The Fix buttons re-route the affected links and remove both collisions.

8.3 What cannot be fixed

- crossings forced by non-planarity (section 2.2);
- a vertex whose required directions exceed the free ones (eight in total, seven at a non-root vertex besides its parent edge);
- crossings that would need a change outside the search space of the fixers, e.g. moving a single vertex independently of its subtree, or choosing a different spanning tree.

9 Time and space complexity

Let n be the number of vertices and m the number of edges. The degree bound gives $m \le 4n$, the drawing has at most $2m \le 8n$ segments, and there are $k = m - n + c \le m$ cycle edges. The beam width $B \le 4$ and the number of options per child $|O| \le 12$ are constants.

1. **Checks and spanning forest:** degree check, BFS forest and cycle-edge classification take $O(n+m)$.
2. **Tree links:** as in the tree algorithm, each of the $n-c$ children is evaluated in $B|O|$ candidates; a candidate is checked against the geometry near its parent and its average distance to all placed vertices is computed, $O(n+m)$ per candidate. Total $O(B|O|\,n(n+m)) = O(n^2)$.
3. **Cycle edges:** a cycle edge is routed in every candidate of its second endpoint, at most $B|O|$ times, with 2 shapes, each checked against all placed vertices and all drawn segments, $O(n+m)$. Total $O(B|O|\,k\,(n+m)) = O(n^2)$ since $k \le m \le 4n$. The pull term costs $O(\deg v)$ per candidate.
4. **One layout:** $O\big(B|O|\,(n+k)(n+m)\big) = O(n^2)$ time and $O(n+m) = O(n)$ space (positions, bends, segments, port sets; the beam holds at most $B|O|$ partial assignments of one vertex's children).
5. **Quality evaluation:** all vertex pairs and all segment pairs, $O(n^2+m^2) = O(n^2)$ (most segment pairs are rejected by a bounding-box test in constant time).
6. **Settings search:** at most 23 layouts with their evaluations, $O(n^2)$ with a constant factor of at most 23; afterwards the result is cached.

7. **Step by Step BFS** (section 7.1): the first click computes the layout, $O(n^2)$ (the settings search is cached if the layout was run before). Every further click compares the new vertex with the drawn vertices, $O(n)$, and its at most 8 new edges (at most 2 segments each) with the drawn segments, $O(m)$, and rewrites both lists, $O(n+m)$; redrawing the visible edges is $O(m)$. Merging groups costs time linear in their sizes; since there are at most $n-1$ merges of vertex groups and $m-1$ of edge groups, each of cost at most n resp. m, all merges of a run together cost $O(n^2+m^2)$. A complete click-through therefore takes $O(n\cdot(n+m)+n^2+m^2) = O(n^2)$ time, the same order as Run Full Metro Layout, and $O(n+m)$ space. (Recomputing the lists from scratch after every click, as in the first version, costs $\Theta(k^2)$ in click k and $\sum_k k^2 = \Theta(n^3)$ in total.)
8. **Fix buttons:** at most $\max(150, \min(500, 25\,000/n))$ evaluations of $O(n^2)$ each, i.e. $O(n^2)$ with a large constant (and $O(n)$ space per stored set of overrides).

graph	n	m	time	environment
grid 5×5	25	40	59 ms	Chromium, Run Full Metro Layout
random graph, degree ≤ 4	40	52	159 ms	Chromium
tree with cycle edges	80	91	314 ms	Chromium
tree with cycle edges	300	344	2.3 s	Chromium
tree with cycle edges	150 / 220 / 300		0.8 / 1.4 / 2.1 s	Node.js, incl. settings search
grid 12×12 / 17×17	144 / 289	264 / 544	1.2 / 5.3 s	Node.js
Fix Vertex + Fix Edge, random graph	40	52	2.4 s	Chromium

Table 1: Measured running times of the first layout of a graph (including the settings search) and of the Fix buttons.

n (tree with cycle edges)	m	recomputing the lists ($O(n^3)$)	incremental ($O(n^2)$)	speed-up
80	91	1.3 s	0.17 s	8×
150	172	5.5 s	0.56 s	10×
220	252	21.2 s	1.7 s	12×
300	344	38.4 s	2.3 s	17×

Table 2: Time for a complete Step by Step BFS click-through after the first click (Chromium, $\alpha = 0.75$): the earlier version, which recomputed the layout and all pairs after every click, and the incremental version. The ratio grows roughly linearly with n, as expected from $O(n^3)$ against $O(n^2)$.

10 Experiments

Five sets of test graphs were laid out with root A: 12 trees with about 15% additional edges between vertices at BFS distance 2–3 ($n = 20, 40, 80, 150$, at most 3 children per vertex); 6 grids from 3×3 to 10×12; the cycles $C_5, C_6, C_{12}, C_{30}, C_{60}$; 6 random graphs with degree ≤ 4 ($n = 20, 40, 80$, $m \approx 1.3n$); and 4 random graphs with degree ≤ 8 ($n = 30, 60$, $m \approx 1.8n$). The graphs are provided as `graph_experiment_sets.zip` (adjacency lists, with an index of their sizes and results). The experiment used each vertex's neighbours in the order in which the generator created them; the page's list importer builds the neighbour lists in a different order, which changes the BFS spanning tree slightly, so per-graph results after importing a file can differ a little from the table (for example 39 instead of 44 colliding edge pairs for the first set at $\alpha = 0.75$); the index lists both. Three variants are compared: *naive* (cycle edges routed, but their route cost is not added to the placement score; the pull is kept; no settings search), *no search* (cycle-aware placement, single run) and the *final* algorithm.

set	α	naive	no search	final
trees + cycle edges (12)	1.0	0/140 (2)	0/65 (3)	0/39 (4)
	0.75	8/126 (1)	4/75 (2)	0/44 (3)
	0.5	17/133 (2)	15/77 (3)	0/45 (3)
grids (6)	1.0	0/497 (0)	0/0 (6)	0/0 (6)
	0.75	0/944 (0)	0/3 (4)	0/0 (6)
	0.5	0/707 (0)	1/51 (3)	0/0 (6)
cycles (5)	1.0	0/0 (5)	0/0 (5)	0/0 (5)
	0.75	0/3 (2)	0/0 (5)	0/0 (5)
	0.5	0/3 (2)	0/0 (5)	0/0 (5)
random, degree ≤ 4 (6)	1.0	0/429	0/334	0/178
	0.75	2/325	5/267	0/178
	0.5	6/369	2/340	0/192
random, degree ≤ 8 (4)	1.0	0/1188	0/881	0/611
	0.75	3/1044	4/1142	0/676
	0.5	17/1072	21/986	1/686

Table 3: Overlapping vertex pairs / colliding edge pairs, summed over each set, and in brackets the number of graphs drawn without any overlap (random graphs: none in any variant). Every edge in every layout was checked to be straight or to have exactly one 45° bend.

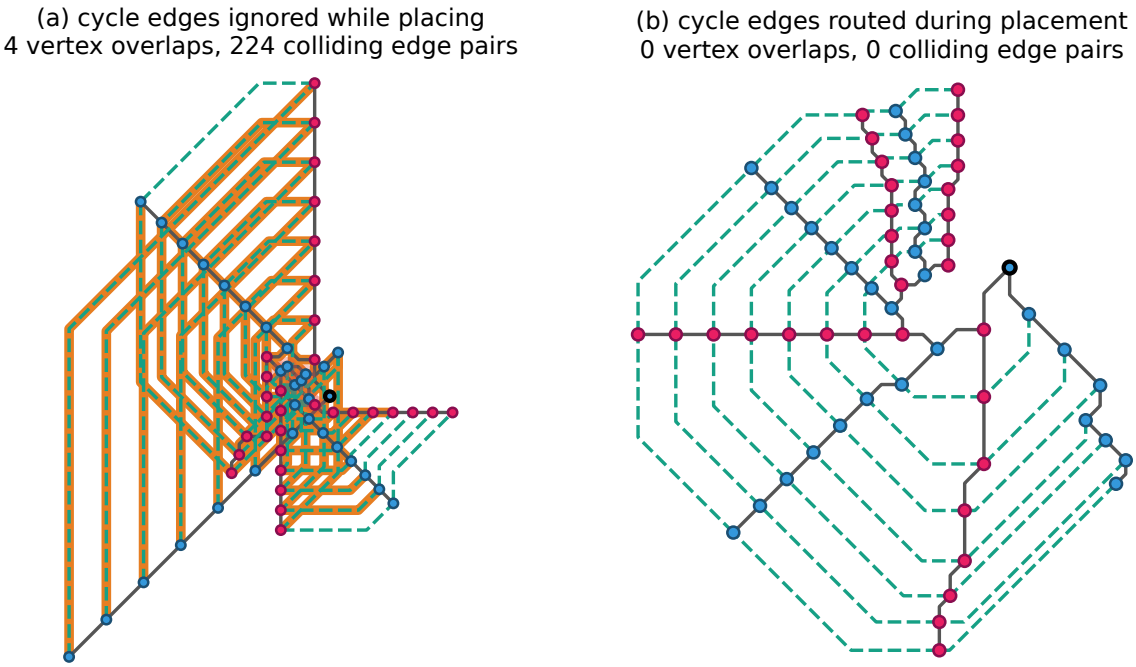


Figure 8: An 8×8 grid ($n = 64$, $m = 112$, 49 cycle edges), $\alpha = 0.75$. (a) If cycle edges are ignored while the vertices are placed (no route cost, no pull, no settings search) and only drawn afterwards, the BFS tree spreads the grid out and the cycle edges collide massively (orange). (b) With the route cost and the pull in the placement score the grid is drawn without any overlap.

Observations: routing cycle edges during placement is the decisive step (grids: hundreds of collisions become none); the settings search removes the remaining vertex overlaps and lowers edge collisions further, especially for small α. Random graphs keep collisions: many of them are not planar, and the spanning-tree-first strategy cannot rearrange the drawing globally. The Fix buttons improve them further (e.g. 15 to 6 colliding edge pairs on a 40-vertex random graph).

11 Limitations

- **Heuristic.** The method gives no guarantee of a crossing-free drawing even for planar graphs and does not minimise the number of crossings; non-planar graphs keep at least $\mathrm{cr}(G)$ crossings.
- **Fixed spanning tree.** The BFS tree from the chosen root decides which edges are tree links; another root or another spanning tree can give a better drawing.
- **Cycle edges have free lengths.** Their length is set by their endpoints, not by α; the gender rule applies to tree links only.
- **Ports of cycle edges** follow from the geometry; with many cycle edges at one vertex, direction conflicts are possible even when $\deg \leq 8$.
- **Size.** The page draws at most 300 vertices; the first layout of a large graph with cycles can take a few seconds.

Reference. J. Korst, V. Pronk and J. J. van Wijk. A Visualization of Family Relations Inspired by the London Metro Map. In *VINCI 2020*, Eindhoven, The Netherlands. ACM, 2020. https://doi.org/10.1145/3430036.3430065